

À propos de la Programmation Objet

Fabrice.Kordon@lip6.fr



Motivation de cette séquence

2

Rajoutée après plusieurs années d'observations

- Constat, vous savez écrire du code linéaire
- Constat, vous ne savez pas le répartir au sein des classes

Résultat

- Les méthodes ne sont pas au «bon endroit»
- Non-respect du modèle objet

Conséquences...

- En programmation séquentielle : «moche» et peu maintenable
- En programmation concurrente : inexécutable

Motivation de cette séquence

2

Rajoutée après plusieurs années d'observations

- Constat, vous savez écrire du code linéaire
- Constat, vous ne savez pas le répartir au sein des classes

Résultat

- Les méthodes
- Non respect



Objectif de la séquence

Proposer quelques «règles types»

Conséquences...

- En programmation séquentielle : «moche» et peu maintenable
- En programmation concurrente : inexécutable

Règle 1

3

 **Les accesseurs, dans la même classe que les attributs qui y sont définis, tu mettras**



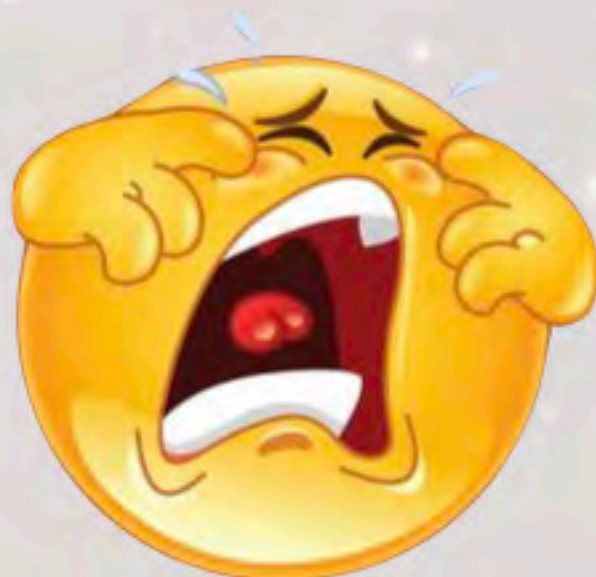
 **Raison**

-  Les accesseurs («setten» et «getten») permettent l'accès aux données de la classe
-  Notion d'encapsulation

Application de la règle 1

4

```
class C1 {  
    int MonAttribut = 0;  
}  
  
class C2 {  
    C1 att = new C1();  
  
    int getMonAttribut () {  
        return att.MonAttribut;  
    }  
  
    void setMonAttribut (int val) {  
        att.MonAttribut = val;  
    }  
}
```



```
class C1 {  
    int MonAttribut = 0;  
  
    int getMonAttribut () {  
        return this.MonAttribut;  
    }  
  
    void setMonAttribut (int val) {  
        this.MonAttribut = val;  
    }  
}
```



Règle 2

5

 Les attributs, autant que possible, tu protégeras

Raison

-  Garantir l'encapsulation malgré les fautes d'inattention
-  Meilleure évolutivité du code
-  Le compilateur/interpréteur indiquera les erreurs



Application de la règle 2

6

```
class C1 {  
    int MonAttribut = 0;  
    int getMonAttribut () {  
        return this.MonAttribut;  
    }  
    void setMonAttribut (int val) {  
        this.MonAttribut = val;  
    }  
}
```

```
class C1 {  
    private int MonAttribut = 0;  
    int getMonAttribut () {  
        return this.MonAttribut;  
    }  
    void setMonAttribut (int val) {  
        this.MonAttribut = val;  
    }  
}
```



Protection contre les
fautes d'inattention
+
aide du compilateur



Application de la règle 2

6

```
class C1 {
```

```
    int MonAttribut
```

```
    int getMonAtt
```

```
        return thi
```

```
    }
```

```
    void setMonAttr
```

```
        this.MonAtt
```

```
    }
```

```
}
```

```
class C1 {
```

```
    MonAttribut = 0;
```

```
    MonAttribut () {
```

```
        this.MonAttribut;
```

```
    }
```

```
    MonAttribut (int val) {
```

```
        this.MonAttribut = val;
```

```
    }
```

```
}
```

Remarque

private, protected, fileprivate, etc.

Conseil

toujours déclarer vos attributs «private»



Règle 3

🏆 À la localisation d'une méthode dans une classe, tu réfléchiras

🏆 Raison

- Les méthodes modifient l'état d'une classe
- Elles sont intrinsèquement liées à la classe dont elles modifient l'état



Règle 4

8

🏆 À la hiérarchisation des classes,
tu réfléchiras

🏆 Raison

- Un programme non objet = structuration en «couches»
- Un programme objet = structuration en graphe acyclique



En guise de conclusion...

9

Cela devait être dit!

- Trop d'erreurs dans ce sens observées ces dernières années

Mémorisez ces règles!

- Elles vous serviront toute votre vie!
- Non respect en séquentiel = bugs + soucis de maintenance
- Non respect en concurrent = cela ne fonctionnera pas

